

Introduction

This tutorial aims to extract theta-nested Spectral Components (tSCs) as described in the following article:

Vítor Lopes-dos-Santos, Gido M. van de Ven, Alexander Morley, Stéphanie Trouche, Natalia Campo-Urriza and David Dupret (2018) **Parsing hippocampal theta oscillations by nested spectral components during spatial exploration and memory-guided behavior**. Neuron. DOI: [10.1016/j.neuron.2018.09.031](https://doi.org/10.1016/j.neuron.2018.09.031)

Please send any comments or suggestions to Vítor (vmldossantos@pharm.ox.ac.uk) or to David (david.dupret@pharm.ox.ac.uk).

This framework uses Python 2 modules, and you will need two .py scripts: `tSC.py` and `EMDlib.py`. We also provide exemple LFP data so you can try our functions.

Installation (Linux)

After downloading the tSC.zip file, extract it in any folder. Then, navigate to that folder and run the following command.

```
pip install -r requirements.txt
```

LFP data format

The signal of each LFP channel must be stored in a binary file that can be loaded using `numpy.load`. Filename structure is:

```
[basename]_[session number].eeg.[channel number]
```

For example: `mst158-1809-0124_2.eeg.4` for

basename: `mst158-1809-0124`

session number: `2`

channel number: `4`

tSC extraction

In order to run this implementation from Linux terminal command line, first make sure the two required .py files (`tSC.py` and `EMDlib.py`) are in the same folder.

First, go in the folder containing the py files and run:

```
$ python2 tSC.py [data folder] [basename]
```

`[data folder]` specifies where you stored the LFP data.

As an example (assuming the data is in the folder `/data/`):

```
$ python2 tSC.py /data/mst158-1809-0124
```

Note that by running the command line above, you won't be specifying which sessions and channels to process. In that case, the algorithm will run all sessions and channels corresponding to the `[data folder] [basename]` input.

In other words, any file matching `[data folder] [basename]_[session number].eeg.[channel number]` will be processed.

If you want to run specific sessions and channels, you should do:

```
$ python2 tSC.py [data folder] [basename] ses=s1,s2,... ch=ch1,ch2,...
```

As an example:

```
$ python2 tSC.py /data/mst158-1809-0124 ses=2,4,6 ch=4
```

The line above would process sessions 2, 4 and 6 and channel 4; which will be found in the following files:

```
/data/mst158-1809-0124_2.eeg.4
```

```
/data/mst158-1809-0124_4.eeg.4
```

```
/data/mst158-1809-0124_6.eeg.4
```

Importantly, the data from different sessions will be combined for tSC extraction¹. If you want to do it for each session independently, you will need to call tSC.py for each of them.

Function outputs

The algorithm will write the outputs in the same folder of LFP data.

In total, you will find 8 output files for each session and channel.

All file names will have the structure: `[basename]_[session number].[affix].[channel number]`

`[affix]` determines which information is carried by that file, as explained below:

Affix	Description
emd	<code>numpy.ndarray</code> ; Rows are different IMFs and columns are time samples.
emd.ia	<code>numpy.ndarray</code> ; Rows are different IMFs and columns are time samples. Shows instantaneous amplitude of the IMFs in emd file.
emd.if	<code>numpy.ndarray</code> ; Rows are different IMFs and columns are time samples. Shows instantaneous frequency of the IMFs in emd file.
theta.cycles	<code>numpy.ndarray</code> (shape = n, 6); Rows correspond to different theta cycles. Columns correspond to different time references of each theta cycles, in order: zero-crossing, peak, zero-crossing, trough, zero-crossing, peak.
theta.phase	<code>numpy.ndarray</code> ; carries theta phase (radians, 0 to 2π) for each time sample. Intervals with no detected theta cycles are filled with <code>numpy.nan</code> .
theta.cycleamp	<code>numpy.ndarray</code> ; carries theta phase (radians, 0 to 2π) for each time sample.

The file with `'tSC.ica'`² as affix is a dictionary variable with keys described below³.

Key	Description
tSCs	<code>numpy.ndarray</code> ; Carries the tSC vectors. Rows correspond to frequencies and columns correspond to different tSCs.
freqs	<code>numpy.ndarray</code> ; indicate the frequency (in Hz) of each row in tSCs.
mainfreqs	<code>numpy.ndarray</code> ; indicate the peak frequency (in Hz) of each tSC (same order as columns in tSCs key).
cyclePSDs	<code>numpy.ndarray</code> ; Carries the spectral signature of each theta cycle. Rows correspond to the theta cycles from <code>'theta.cycles'</code> and columns correspond to frequencies in <code>freqs</code> key.
projections	<code>numpy.ndarray</code> ; Carries tSC strengths for each tSC in each theta cycle. Rows correspond to theta cycles from <code>'theta.cycles'</code> and columns correspond to tSCs in the tSCs key.

¹ Combining multiple sessions is recommended as the more cycles you provide to the algorithm, the better outcomes you will have.

² In case user chooses to use NMF instead of ICA, the affix will be `'tSC.nmf'`.

³ Keys not mentioned in this table will be used in future developments of this module.

The file with 'tSC.ica.model' as affix is a sklearn-object carrying the model. This can be used if one wants to extract tSCs in a session and then project them in another session.

We also provide a function to generate a theta phase-gamma coupling analysis plot for different sets of theta cycles, as in Figure 1 of Lopes-dos-Santos et al, Neuron 2018.

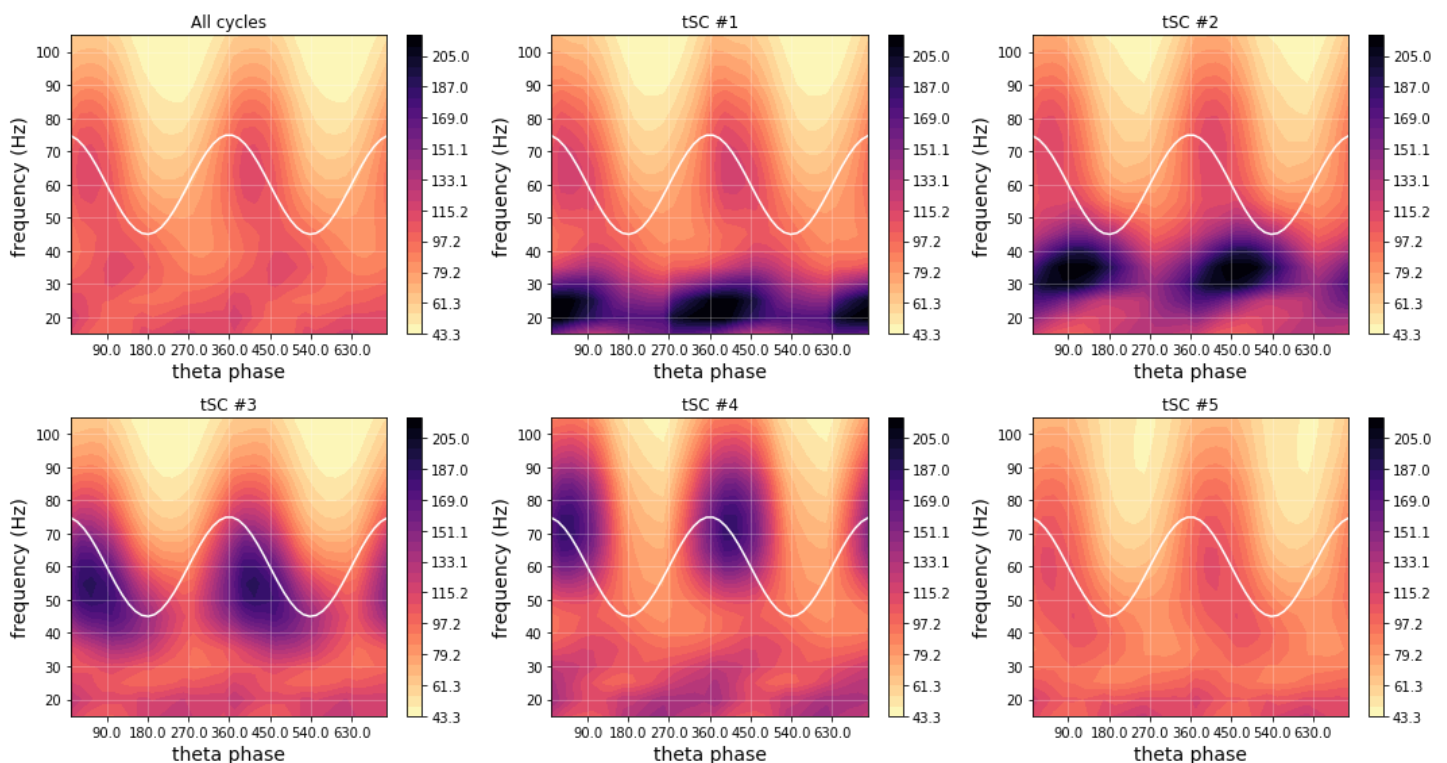
For that, as an example, after running tSC.py as shown above, you can run the following code in Jupyter Notebook:

```
import os as os
os.chdir('/codesdir/tSC/') #set here where the codes are
import tSC as tsc

import matplotlib.pyplot as plt
%matplotlib inline

base = '/data/mst158-1809-0124'
tsc.plotMeantSCcycle(base, ChLabel=4, sessions=[2, 4, 6])
```

You should see the following figure:



Changing default parameters

The parameters you might need/want to set are:

1. **Spectrogram frequencies.** This will set which Morlet wavelet main frequencies are used to compute the spectrogram of the supra-theta signal. This will also determine which frequencies are used for defining spectral signatures and tSC vectors (see Lopes-dos-Santos et al 2018 Introduction and STAR methods for more details). By default, the algorithm will use the set of frequencies used in our study: 10 to 200 Hz in 1-Hz steps. These can be set as follows:

```
$ python2 tSC.py [data folder] [basename] freqs=f1,f2,f3...
```

2. **Number of components.** In our study, 5 components were sufficient to capture at least 85% of the total variance of spectral signatures in each of our recordings (see Lopes-dos-Santos et al 2018 STAR methods). The user can however change the number of spectral components as shown below:
`$ python2 tSC.py [data folder] [basename] nPCs=npcs`
3. **Dimensionality reduction method.** The algorithm currently uses ICA as default to extract tSCs⁴. The code also allows the users to use Non-negative Matrix Factorization (NMF) if they wish. For that, you should run:
`$ python2 tSC.py [data folder] [basename] dimrec=nmf`
4. **LFP sampling rate.** If not set, algorithm will assume the sampling rate of the recorded LFPs is 1250 Hz, as it is the case of the provided exemplary data. In order to change it , you should run:
`$ python2 tSC.py [data folder] [basename] sr=sr`
5. **Number of workers for eEMD parallelization.** The ensemble Empirical Mode Decomposition (eEMD) is run in parallel. As default, the algorithm will use one third of the available CPUs, but this can be changed by:
`$ python2 tSC.py [data folder] [basename] nCPUs=nCPUs`

Thus, as an example, if one runs:

```
$ python2 tSC.py /data/mst158-1809-0124 ses=2,4,6 ch=4 nPCs=4 dimrec=nmf sr=1000  
nCPUs=40
```

This will process sessions 2, 4 and 6 (combined), channel 4, with 4 components, using non-negative matrix factorization, assuming 1000 Hz sampling rate and it will use 40 workers for parallelizing eEMD.

We hope you will find this tool and its tutorial useful.

⁴ The conceptual rationale behind the employment of ICA was to extract spectral components reflecting statistically independent sources, and thus likely representing different oscillations.